

PRÁCTICA 2: SPARK STRUCTURED STREAMING

Procesamiento de flujos de datos estructurados con Kafka y PySpark

SISTEMAS DISTRIBUIDOS DE PROCESAMIENTO DE DATOS II
Curso 2025/2026

Índice

1. Introducción	3
2. Carga de datos en Apache Kafka.....	3
2.1. Productor Kafka.....	3
2.2. Consumidor Kafka.....	4
3. Lectura del flujo con Spark Structured Streaming.....	5
4. Respuestas razonadas a las preguntas	8
4.1. Pregunta 1	8
4.2. Pregunta 2.....	8
4.3. Pregunta 3.....	9
5. Consultas implementadas sobre el stream	9
5.1. Consulta 1: conteo de compras por producto	9
5.2. Consulta 2: filtrado de productos	10
6. Valoración final del trabajo.....	11
7. Anexo: scripts utilizados	12

1. Introducción

En esta segunda parte de la práctica se ha trabajado con Apache Spark Structured Streaming para procesar datos en forma de flujo a partir de un topic de Apache Kafka. La práctica continúa la línea de trabajo de la primera fase, donde ya se había utilizado Kafka como sistema de mensajería, pero en este caso el objetivo principal es comprender cómo Spark puede leer esos datos como un stream estructurado y ejecutar consultas sobre ellos.

El trabajo se divide en dos partes principales. En primer lugar, se cargan datos de ejemplo en el topic purchases mediante un productor de Kafka y se comprueba su lectura con un consumidor en Python. En segundo lugar, se utiliza PySpark para leer el flujo desde ese mismo topic, crear una tabla temporal en memoria y aplicar consultas básicas sobre los mensajes recibidos.

A lo largo del documento se incluyen las capturas de terminal solicitadas, junto con una explicación de cada fase del proceso y las respuestas razonadas a las preguntas del enunciado.

2. Carga de datos en Apache Kafka

En el primer apartado de la práctica se preparan los datos de ejemplo y se insertan en una cola de Apache Kafka. Para ello se han utilizado dos scripts: `kafka-producer-confluent_g2.py`, que actúa como productor de mensajes, y `kafka-consumer-confluent_g2.py`, que permite comprobar que los mensajes se reciben correctamente desde el topic.

2.1. Producer Kafka

El productor se conecta al broker de Kafka configurado para el grupo y publica mensajes aleatorios en el topic purchases. Cada mensaje contiene una clave, que representa el identificador de usuario, y un valor, que representa el producto comprado. En la captura se observa que el entorno virtual `kafka_env` está activado y que el script genera varios eventos con productos como book, t-shirts, batteries, alarm clock o gift card.

```
vboxuser@SDPD2-lab: ~/Escritorio/practica2
vboxuser@SDPD2-lab:~/Escritorio/practica2$ source ~/kafka_env/bin/activate
(kafka_env) vboxuser@SDPD2-lab:~/Escritorio/practica2$ python3 kafka-producer-confluent_g2.py
Produced event to topic purchases: key = htanaka-2      value = book
Produced event to topic purchases: key = jsmith-2       value = t-shirts
Produced event to topic purchases: key = jbernard-2     value = batteries
Produced event to topic purchases: key = awalther-2     value = alarm clock
Produced event to topic purchases: key = htanaka-2      value = book
Produced event to topic purchases: key = jbernard-2     value = gift card
Produced event to topic purchases: key = jsmith-2       value = gift card
Produced event to topic purchases: key = htanaka-2      value = t-shirts
Produced event to topic purchases: key = jbernard-2     value = batteries
Produced event to topic purchases: key = jsmith-2       value = t-shirts
```

Figura 1. Ejecución del productor Kafka y publicación de eventos en el topic purchases.

Esta salida demuestra que el productor consigue enviar correctamente los mensajes al topic. Además, al imprimirse por pantalla la clave y el valor de cada evento, se puede comprobar de forma directa qué datos se están insertando en Kafka.

2.2. Consumer Kafka

Después de producir los mensajes, se ejecuta el consumidor Kafka para verificar que los eventos insertados pueden leerse sin errores. El consumidor se suscribe al topic purchases y va mostrando por terminal los mensajes consumidos, indicando también su clave y su valor.

```
vboxuser@SDPD2-lab: ~/Escritorio/practica2
Waiting...
Waiting...
Waiting...
Consumed event from topic purchases: key = htanaka-2    value = book
Consumed event from topic purchases: key = jsmith-2     value = t-shirts
Consumed event from topic purchases: key = jbernard-2   value = batteries
Consumed event from topic purchases: key = awalther-2   value = alarm clock
Consumed event from topic purchases: key = htanaka-2    value = book
Consumed event from topic purchases: key = jbernard-2   value = gift card
Consumed event from topic purchases: key = jsmith-2     value = gift card
Consumed event from topic purchases: key = htanaka-2    value = t-shirts
Consumed event from topic purchases: key = jbernard-2   value = batteries
Consumed event from topic purchases: key = jsmith-2     value = t-shirts
Consumed event from topic purchases: key = awalther-2   value = gift card
Consumed event from topic purchases: key = jbernard-2   value = book
Consumed event from topic purchases: key = eabara-2     value = t-shirts
Consumed event from topic purchases: key = eabara-2     value = book
Consumed event from topic purchases: key = sgarcia-2    value = book
Consumed event from topic purchases: key = awalther-2   value = batteries
Consumed event from topic purchases: key = jbernard-2   value = book
Consumed event from topic purchases: key = jsmith-2     value = book
Consumed event from topic purchases: key = sgarcia-2    value = gift card
Consumed event from topic purchases: key = jbernard-2   value = t-shirts
```

Figura 2. Ejecución del consumidor Kafka leyendo mensajes desde el topic purchases.

En la captura se observa que el consumidor recibe correctamente los mensajes publicados previamente. Esto permite validar que el proceso de comunicación con Kafka funciona de forma adecuada antes de pasar a la parte de Spark Structured Streaming.

3. Lectura del flujo con Spark Structured Streaming

Una vez comprobado que Kafka recibe y entrega los mensajes correctamente, se pasa a la lectura del flujo desde Spark. Para ello se utiliza el script `struct_kafka_consumer_local_g2.py`, ejecutado con el entorno virtual `pyspark-411`. Este programa crea una sesión de Spark en modo local, carga el conector Spark-Kafka y lee los mensajes del topic purchases como un DataFrame de streaming.

```
.readStream
.format("kafka")
.option("kafka.bootstrap.servers", "localhost:9092")
.option("subscribe", "purchases")
.option("startingOffsets", "earliest")
.load()
```

El valor del mensaje de Kafka se transforma a tipo string mediante `selectExpr("CAST(value AS STRING)")`, ya que los datos recibidos desde Kafka llegan inicialmente en formato binario. Después, el stream se escribe en memoria con el nombre `raw_data`, lo que permite realizar consultas SQL sobre él durante la ejecución del programa.

```
describe_query = (input_data.writeStream
    .queryName("raw_data")
    .format("memory")
    .outputMode("append")
    .start())
```

El uso de `.format("memory")` permite que el resultado del stream se almacene temporalmente en memoria dentro de la sesión de Spark. Además, mediante `.queryName("raw_data")`, Spark registra automáticamente una tabla temporal con ese nombre en el catálogo interno del `SparkSession`.

Gracias a esto, los datos recibidos desde Kafka pueden consultarse directamente utilizando sentencias SQL como:

```
spark.sql("SELECT * FROM raw_data")
```

Esto hace que el stream pueda tratarse de forma muy similar a una tabla convencional, aunque en realidad los datos se van actualizando dinámicamente conforme llegan nuevos micro-batches desde Kafka.

```
=====
Spark Structured Streaming - Grupo 2
Broker : localhost:9092
Topic  : purchases
=====
26/05/05 12:09:23 WARN ResolveWriteToStream: Temporary checkpoint location creat
ed which is deleted normally when the query didn't fail: /tmp/temporary-9a2d3bc5
-eb41-403c-9954-d41021979387. If it's required to delete it under any circumstan
ces, please set spark.sql.streaming.forceDeleteTempCheckpointLocation to true. I
mportant to know deleting temp checkpoint folder is best effort.
26/05/05 12:09:23 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not s
upported in streaming DataFrames/Datasets and will be disabled.

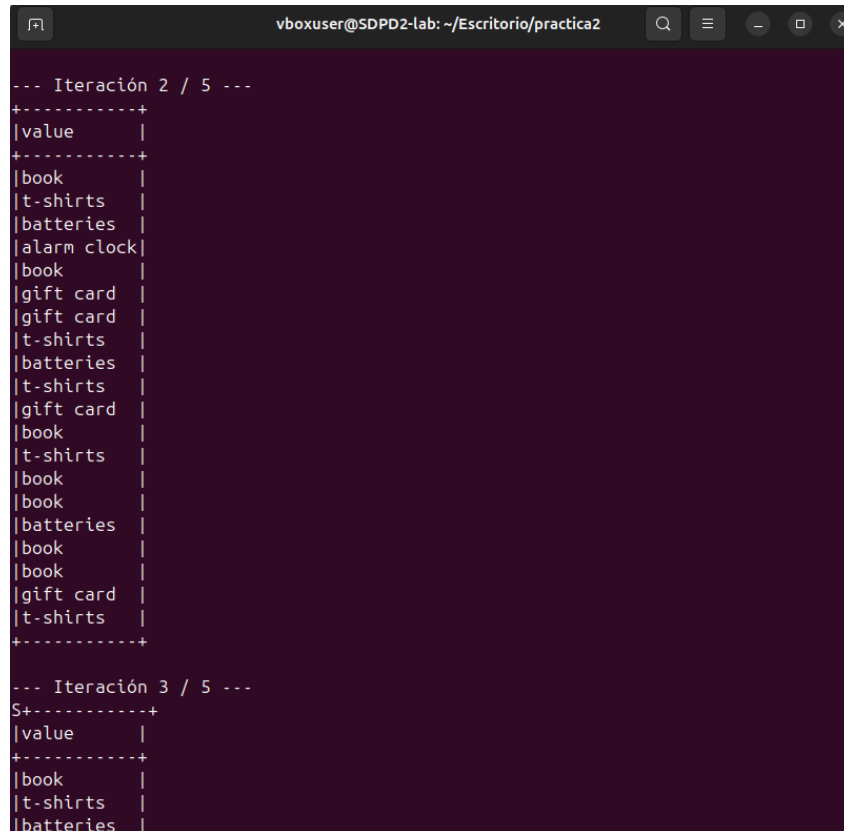
[raw_data] Stream iniciado. Monitorizando durante 5 iteraciones...

--- Iteración 1 / 5 ---
+-----+
|value|
+-----+
+-----+

--- Iteración 2 / 5 ---
+-----+
|value      |
+-----+
|book       |
|t-shirts   |
|batteries  |
|alarm clock|
|book       |
|gift card  |
|gift card  |
|t-shirts   |
|batteries  |
```

Figura 3. Inicio del programa de Spark Structured Streaming y primera lectura de la tabla `raw_data`.

En la primera iteración se aprecia que la tabla `raw_data` todavía no contiene valores. Esto ocurre porque la consulta de streaming se lanza de forma asíncrona y Spark aún no ha terminado de conectar con Kafka ni de procesar el primer micro-batch.



```
vboxuser@SDPD2-lab: ~/Escritorio/practica2

--- Iteración 2 / 5 ---
+-----+
|value   |
+-----+
|book    |
|t-shirts|
|batteries|
|alarm clock|
|book    |
|gift card|
|gift card|
|t-shirts|
|batteries|
|t-shirts|
|gift card|
|book    |
|t-shirts|
|book    |
|book    |
|batteries|
|book    |
|book    |
|gift card|
|t-shirts|
+-----+

--- Iteración 3 / 5 ---
S+-----+
|value   |
+-----+
|book    |
|t-shirts|
|batteries|
```

Figura 4. Segunda iteración del stream, donde ya aparecen los primeros productos leídos desde Kafka.

```
vboxuser@SDPD2-lab: ~/Escritorio/practica2
|t-shirts |
+-----+
--- Iteración 5 / 5 ---
+-----+
|value    |
+-----+
|book     |
|t-shirts |
|batteries|
|alarm clock|
|book     |
|gift card|
|gift card|
|t-shirts |
|batteries|
|t-shirts |
|gift card|
|book     |
|t-shirts |
|book     |
|book     |
|batteries|
|book     |
|book     |
|gift card|
|t-shirts |
+-----+

[agg_data] Iniciando consulta de agregación (complete mode)...
26/05/05 12:09:40 WARN ResolveWriteToStream: Temporary checkpoint location created which is deleted normally when the query didn't fail: /tmp/temporary-366bcca6
```

Figura 5. Iteración posterior del stream mostrando los datos acumulados en raw_data.

En las iteraciones posteriores ya aparecen productos como book, t-shirts, batteries, alarm clock y gift card. Esto confirma que Spark ha podido conectarse al topic de Kafka y leer correctamente los mensajes publicados.

4. Respuestas razonadas a las preguntas

4.1. Pregunta 1

¿Por qué no se obtiene ningún valor en la primera iteración que imprime por la terminal el resultado de la consulta a la tabla raw_data?

La primera iteración aparece vacía porque la consulta de Structured Streaming se ejecuta de forma asíncrona. Al llamar a `.start()`, Spark inicia la consulta en segundo plano y devuelve el control al programa principal sin esperar a que el primer micro-batch haya terminado de procesarse.

En ese primer momento Spark todavía está estableciendo la conexión con Kafka, negociando los offsets, leyendo los mensajes iniciales y materializando el resultado en la tabla en memoria `raw_data`. Por eso, cuando se ejecuta la primera consulta `SELECT * FROM raw_data`, la tabla aún no contiene filas.

Después de esperar unos segundos entre iteraciones, Spark ya tiene tiempo de completar el primer micro-batch y la tabla empieza a mostrar los valores recibidos desde Kafka.

4.2. Pregunta 2

¿Qué función tiene la siguiente línea de código para configurar el input stream de Kafka?


```
.option("startingOffsets", "earliest")
```

Esta opción indica desde qué posición del topic de Kafka debe empezar a leer Spark. Al utilizar `earliest`, el stream comienza desde el offset más antiguo disponible en el topic. Es decir, Spark lee también los mensajes que ya estaban publicados antes de arrancar la aplicación.

Esto es importante en la práctica porque primero se ejecuta el productor Kafka y después el consumidor con Spark. Si se usara `latest`, que es el comportamiento habitual para leer solo mensajes nuevos, Spark podría arrancar sin mostrar los mensajes generados previamente. En cambio, con `earliest` se recupera todo el contenido disponible en el topic `purchases` desde el principio.

4.3. Pregunta 3

¿Qué tipo de stream de salida crea el siguiente fragmento de código y por qué podemos consultarlo como si fuera una tabla ordinaria?

```
describe_query = (input_data.writeStream.queryName("raw_data")
                  .format("memory").outputMode("append")
                  .start())
```

Este fragmento crea un stream de salida en memoria, también llamado memory sink. Al usar `format("memory")`, Spark guarda el resultado de la consulta en la memoria de la sesión. Con `queryName("raw_data")`, ese resultado se registra con el nombre `raw_data` dentro del catálogo de `SparkSession`.

Gracias a esto, se pueden hacer consultas directamente con `spark.sql("SELECT * FROM raw_data")`, igual que si se estuviera trabajando con una tabla normal. La diferencia es que esta tabla se va actualizando conforme llegan nuevos micro-batches del stream.

El modo `append` indica que se añaden únicamente las nuevas filas procesadas en cada micro-batch, sin reemplazar los datos anteriores. Este comportamiento encaja con una lectura directa de mensajes de Kafka, donde cada nuevo evento se incorpora al resultado acumulado.

5. Consultas implementadas sobre el stream

Además de consultar directamente los datos crudos recibidos desde Kafka, se implementan dos consultas adicionales sobre el stream. Cada una utiliza un modo de salida distinto de Spark Structured Streaming, tal y como solicita el enunciado.

5.1. Consulta 1: conteo de compras por producto

La primera consulta agrupa los datos por el valor del mensaje, que en esta práctica corresponde al producto comprado, y calcula cuántas veces aparece cada producto en el stream.

```
agg_data = (input_data
            .groupBy("value")
            .agg(F.count("*").alias("total")))
```

Para esta consulta se utiliza `outputMode("complete")`, ya que se trata de una agregación. En este modo, Spark actualiza la tabla completa de resultados en cada micro-batch, mostrando el estado completo de los conteos acumulados.

Este modo de salida resulta necesario en operaciones de agregación, ya que Spark debe recalcular continuamente el resultado global conforme llegan nuevos datos al stream. Por ello, en cada micro-batch se reescribe la tabla completa con los conteos actualizados.

```
agg_query = (agg_data.writeStream
    .queryName("agg_data")
    .format("memory")
    .outputMode("complete")
    .start())
```

```
(pyspark-411) vboxuser@SDPD2-lab:~/Escritorio/practica2$ cat salida1.txt
CONSULTA 1 - Conteo de compras por producto (outputMode=complete)
=====
```

Figura 6. Comprobación del fichero salida1.txt generado para la consulta de agregación.

```
CONSULTA 1 - Conteo de compras por producto (outputMode=complete)
=====
      value  total
      book    13
  batteries     9
    t-shirts     8
  alarm clock     7
    gift card     3
```

Figura7. Contenido del fichero salida1.txt

El resultado de esta consulta se vuelca en el fichero salida1.txt. Esta salida corresponde al conteo de compras por producto utilizando el modo complete, adecuado para consultas con agregación porque el resultado puede cambiar cada vez que llegan nuevos datos.

5.2. Consulta 2: filtrado de productos

La segunda consulta filtra determinados productos del stream. En el script se seleccionan productos como book y alarm clock, y se añade una etiqueta auxiliar para identificar esos mensajes como elementos filtrados.

```
filtered_data = (input_data
    .filter(F.col("value").isin(["book", "alarm clock"]))
    .withColumn("label", F.lit("high_value_item")))
```

En este caso se utiliza outputMode("append"), porque la consulta no necesita mantener un estado agregado. Cada mensaje nuevo se evalúa de forma independiente y, si cumple la condición del filtro, se añade al resultado de salida.

A diferencia de la consulta anterior, esta operación no necesita mantener un estado acumulado. Cada mensaje se procesa de forma independiente y, si cumple la condición del filtro, se añade directamente al stream de salida sin recalcular resultados previos.

```
filtered_query = (filtered_data.writeStream
    .queryName("filtered_data")
    .format("memory")
    .outputMode("append")
    .start())
```

El objetivo de esta segunda consulta es mostrar un ejemplo de operación sin estado sobre el stream. A diferencia del conteo por producto, aquí no se recalcula una tabla completa, sino que simplemente se incorporan las nuevas filas que cumplen el filtro.

CONSULTA 2 - Filtrado productos (outputMode=append)

```
=====
  usuario    producto
awalther-2   book
eabara-2     t-shirts
jsmith-2     alarm clock
jsmith-2     book
jbernard-2   book
eabara-2     book
jsmith-2     alarm clock
jbernard-2   alarm clock
sgarcia-2    book
sgarcia-2    t-shirts
jsmith-2     alarm clock
sgarcia-2    book
eabara-2     t-shirts
sgarcia-2    book
htanaka-2    book
jbernard-2   book
htanaka-2    t-shirts
jsmith-2     book
eabara-2     alarm clock
sgarcia-2    book
jsmith-2     book
jbernard-2   t-shirts
jbernard-2   t-shirts
jsmith-2     t-shirts
sgarcia-2    alarm clock
awalther-2   t-shirts
awalther-2   book
htanaka-2    alarm clock
```

Figura 8. Contenido del fichero salida2.txt

Esta salida muestra los registros que han superado el filtro aplicado en el stream, manteniendo tanto el identificador del usuario como el nombre del producto.

La captura de la figura 8 corresponde al filtrado de productos utilizando el modo append, adecuado para consultas sin agregación. Al contrario que en el apartado 5.1, ahora cada registro que cumple la condición se añade al flujo de salida de forma independiente (sin necesidad de mantener o recalcular resultados previos).

6. Valoración final del trabajo

En conjunto, esta práctica permite entender el flujo completo desde la publicación de mensajes en Kafka hasta su consumo mediante Spark Structured Streaming. Primero se comprueba que el productor introduce correctamente eventos en el topic purchases y que el consumidor de Kafka puede leerlos. Después, se enlaza Spark con Kafka para tratar esos mensajes como un flujo de datos estructurado.

Una parte importante del trabajo ha sido observar el funcionamiento por micro-batches. La primera iteración vacía de raw_data ayuda a ver que las consultas de streaming no se comportan igual que una lectura estática, sino que necesitan unos instantes para iniciar la conexión, leer los offsets y materializar los primeros resultados.

También se ha comprobado la diferencia entre los modos de salida append y complete. El modo append resulta adecuado para consultas donde solo se añaden nuevas filas, mientras que complete es necesario en consultas de agregación, donde el resultado global puede cambiar cada vez que entra un nuevo lote de datos.

Como conclusión, la práctica cumple con el objetivo de familiarizarse con Spark Structured Streaming y su integración con Kafka, mostrando tanto la carga y comprobación inicial de los mensajes como la lectura y consulta de un flujo estructurado desde PySpark.

Además, la práctica ha permitido comprender mejor cómo Spark Structured Streaming unifica el procesamiento batch y streaming mediante una misma API, facilitando el tratamiento de flujos de datos como si fueran tablas dinámicas en constante actualización.

7. Anexo: scripts utilizados

Los scripts principales utilizados en la práctica han sido los siguientes:

- `kafka-producer-confluent_g2.py`: productor Kafka encargado de publicar mensajes aleatorios en el topic `purchases`.
- `kafka-consumer-confluent_g2.py`: consumidor Kafka utilizado para comprobar que los mensajes se reciben correctamente.
- `struct_kafka_consumer_local_g2.py`: script principal de Spark Structured Streaming para leer desde Kafka, crear la tabla `raw_data` y ejecutar consultas sobre el stream.
- `generar_salida2.py`: script auxiliar para generar la segunda salida a partir de una lectura desde Kafka.

Estos ficheros complementan el documento y permiten reproducir las ejecuciones descritas en las capturas anteriores.